

Evolution strategies

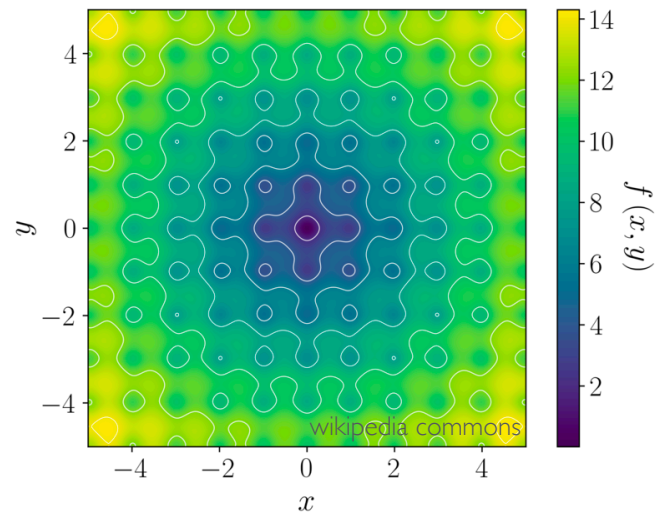
Nicolas Bredeche

Sorbonne Université
ISIR, UMR 7222
Paris, France
nicolas.bredeche@sorbonne-universite.fr



UE AROB - Apprentissage et Robotique
M1 informatique, parcours Androïde

rev. 2025-02-14



Outline

2

- Week 1 : evolution strategies
 - ▶ Concepts and definitions
 - ▶ Population-based methods
 - ▶ Probability-weighted averaging methods
- Week 2 : evolutionary robotics
- Week 3 : collective robotics and learning

Evolution Strategies

Elements

4

problem: $x^* = \operatorname{argopt}_{x \in X} f(x)$

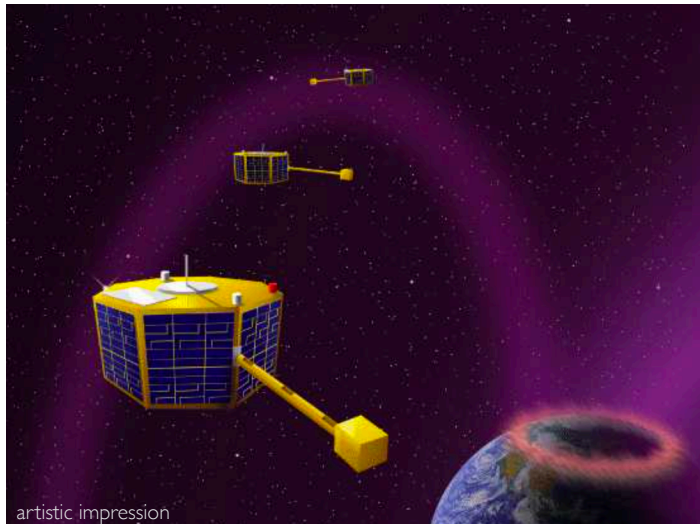
candidate solution: $a := (x, f(x))$

$$f(x) : \mathbb{R}^n \rightarrow \mathbb{R}$$

- **black-box stochastic optimisation** algorithms
 - derivative-free
 - population-based
 - iterative
 - stochastic operators
 - versatile wrt. search space (GA, GP, EP, **ES**, etc.)

$$x^* = \operatorname{argopt}_{x \in X} f(x)$$

- Elements
 - candidate solutions — $x \in X$
 - objective function — f
 - performance value, or “fitness” — $f(x)$
 - objective: maximise (or minimise) the fitness — note: $\max(F) = \min(-F)$



$$f(x) : \mathbb{R}^n \rightarrow \mathbb{R}$$

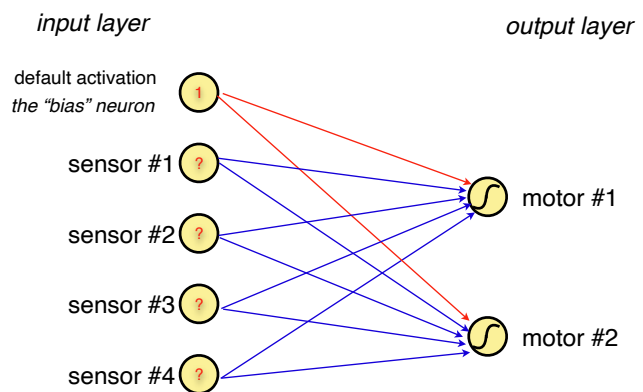
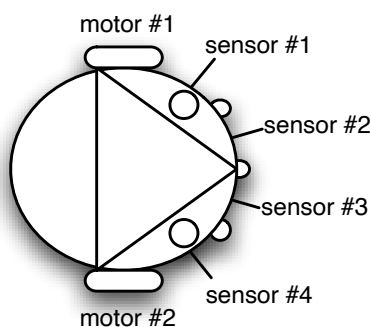
with x , the spatial configuration

$$x^* = \operatorname{argopt}_{x \in X} f(x)$$

the best antenna is obtained w/ parameters x^*

[Horny et al., 2006, NASA] "Automated Antenna Design with Evolutionary Algorithms"

Blackbox optimisation and robotics



NN weights as parameters in $\mathbb{R}^{10}$

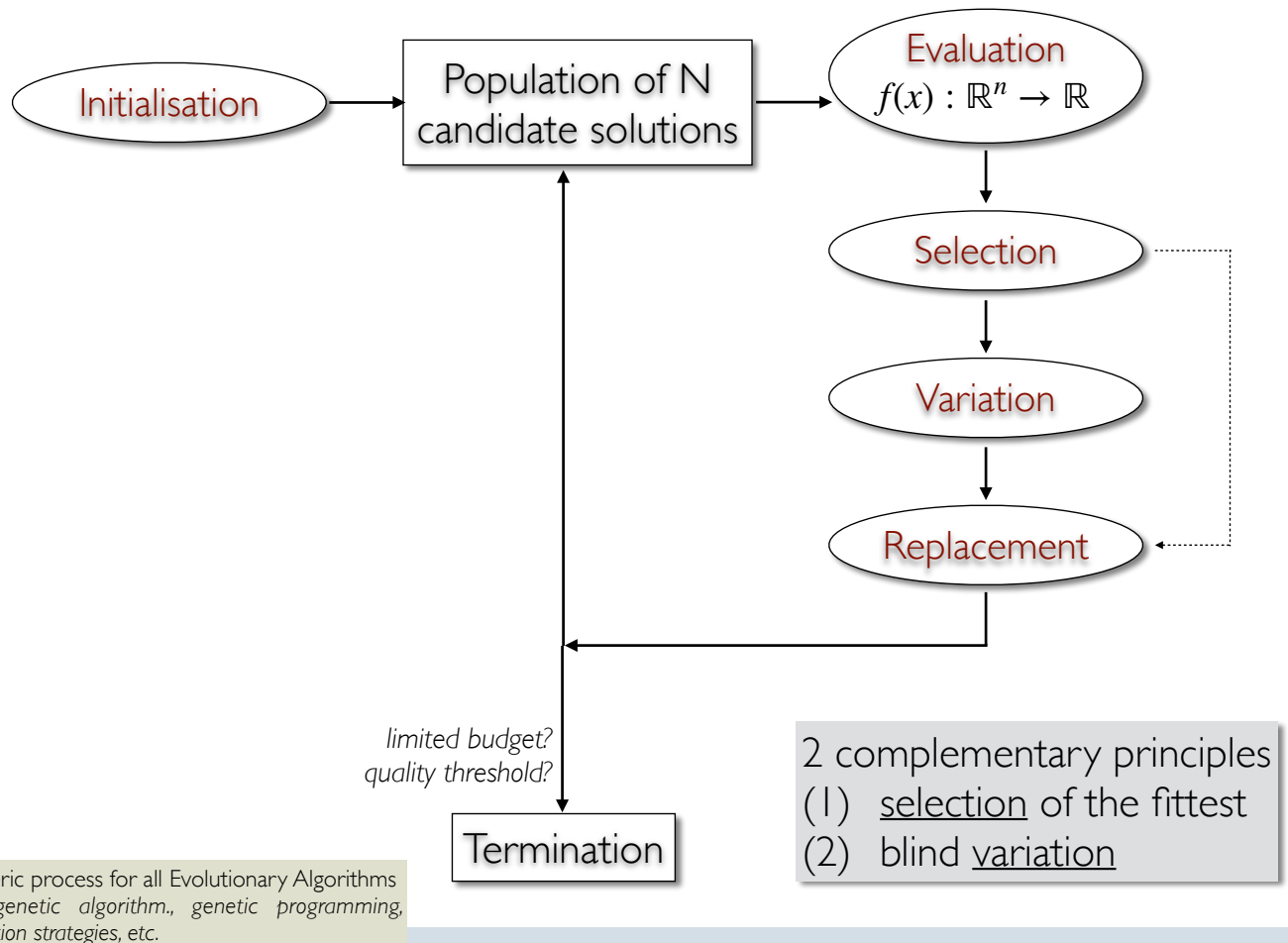
$$\operatorname{output}_j = \tanh\left(\sum_i w_{i,j} * \operatorname{input}_i\right)$$

$$f(x) : \mathbb{R}^n \rightarrow \mathbb{R}$$

with x , the neural network weights

$$x^* = \operatorname{argopt}_{x \in X} f(x)$$

the best trajectory is obtained w/ parameters x^*



- Search space is in $\mathbb{R}^n$
- (μ, λ) notation
 - μ : number of parents ; λ : number of offspring
 - $(\mu + \lambda)$: selection in parents U offsprings
 - (μ, λ) : selection in offsprings only

Evolution Strategies

(1+1)-ES algorithm

Naive (1+1)-ES algorithm

12

init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

for i in range(evaluations):

$x' = x + \sigma N(0, I)$

 if $\text{fitness}(x') \geq \text{fitness}(x)$:

$x = x'$

Details in the next slide...

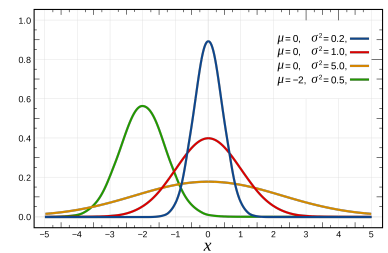
a candidate solution
(the initial "parent")

number of dimensions
(depends on the problem)

init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

mutation step size



for i in range(evaluations):

$x' = x + \sigma N(0, I)$

if $\text{fitness}(x') \geq \text{fitness}(x)$:

$x = x'$

gaussian mutation

Example of implementation:
(...)
for d in dimensions:
 $x'[d] = x[d] + \text{rand.gaussian}(\text{sigma})$
(...)

the "child"

the "parent"

here, we assume this is
a maximisation problem

x : candidate solution, vector of n real values

σ : mutation step-size

$N(0, I)$: multivariate normal distribution with mean=0 and unity covariance matrix

Gaussian : wikipedia commons

init: $x \in \mathbb{R}^n$

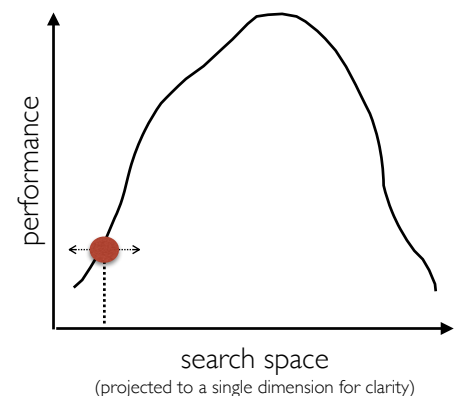
init: $\sigma > 0$

for i in range(evaluations):

$x' = x + \sigma N(0, I)$

if $\text{fitness}(x') \geq \text{fitness}(x)$:

$x = x'$

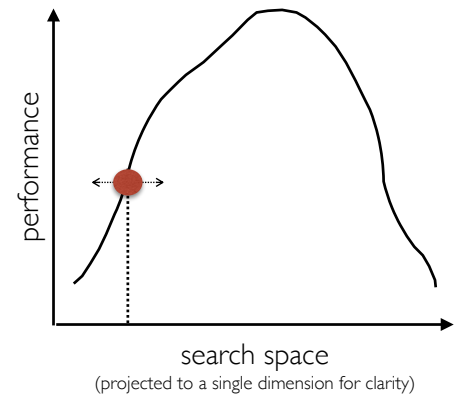


```

init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 

```



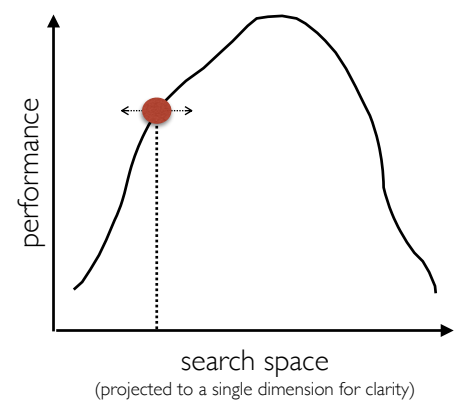
```

init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 

```

sigma is too small!!



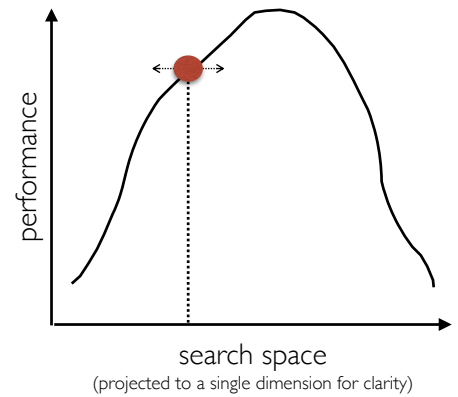

```

init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 

```

sigma is too small!!



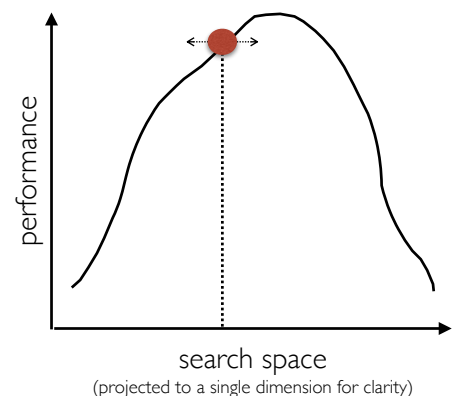
```

init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 

```

sigma is too small!!



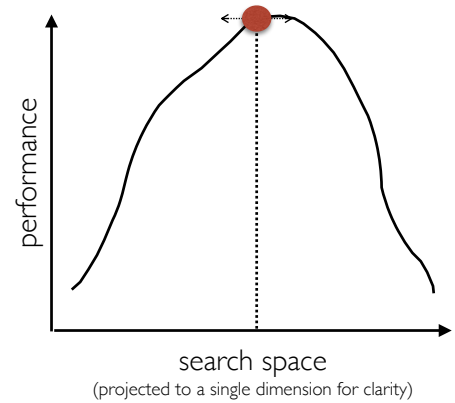
```

init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if  $\text{fitness}(x') \geq \text{fitness}(x)$ :
         $x = x'$ 

```

sigma is too small!

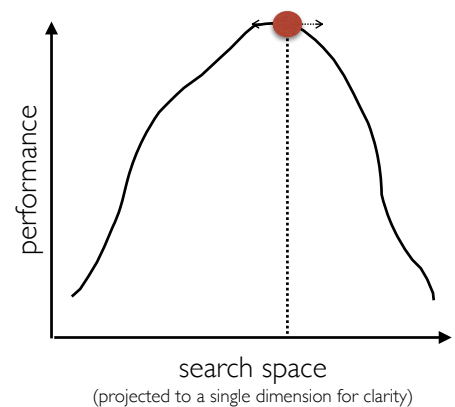


```

init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if  $\text{fitness}(x') \geq \text{fitness}(x)$ :
         $x = x'$ 

```

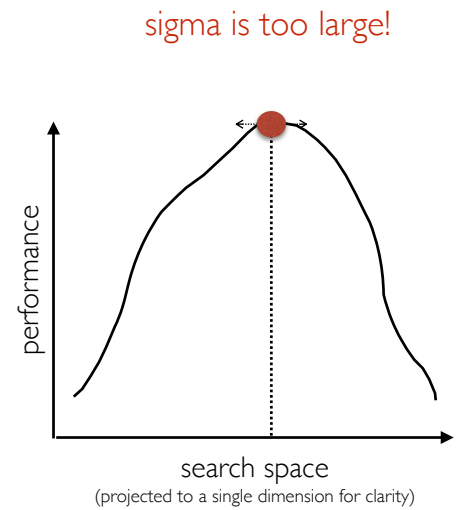


```

init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if  $\text{fitness}(x') \geq \text{fitness}(x)$ :
         $x = x'$ 

```

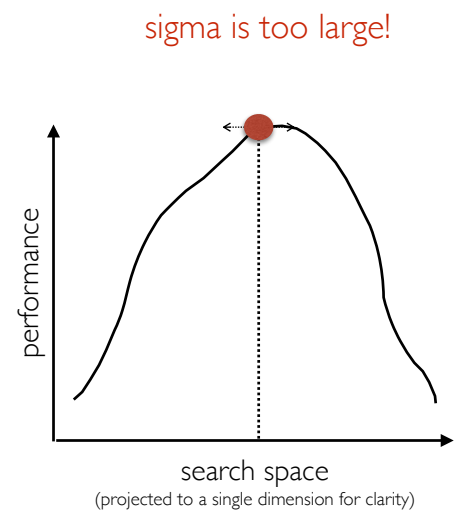


```

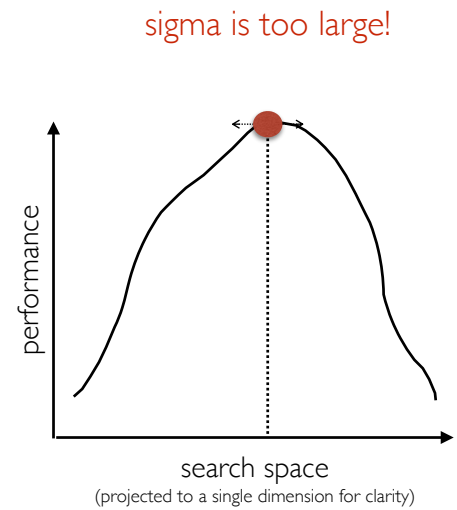
init:  $x \in \mathbb{R}^n$ 
init:  $\sigma > 0$ 

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if  $\text{fitness}(x') \geq \text{fitness}(x)$ :
         $x = x'$ 

```



```
init:  $x \in \mathbb{R}^n$   
init:  $\sigma > 0$   
  
for i in range(evaluations):  
     $x' = x + \sigma N(0, I)$   
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):  
         $x = x'$ 
```



oscillates around the optimum

QUIZZ

Q: l'algorithme (1+1)-ES permet d'optimiser...

1. une forme géométrique définie dans un espace 3D
2. les poids d'un réseaux de neurones
3. une vecteur de valeurs réelles

Q: l'algorithme $(1+1)$ -ES est bien adapté lorsque la fonction objectif est...

1. convexe
2. multimodale
3. unimodale

Q: dans l'algorithme $(1+1)$ -ES, une valeur élevée pour σ permet...

1. d'explorer loin du parent
2. d'explorer proche du parent

Q: lorsqu'on s'approche de l'optimal on préfère avoir...

1. un sigma faible
2. un sigma fort

Q: lorsqu'on construit x' tel que $x' = x + \sigma N(0,1)$, on obtient...

1. un mutant de x dans $[0,1]$
2. un mutant de x dans $[0, \sigma]$
3. un mutant de x tirée d'une gaussienne centrée sur x et d'écart-type 1
4. un mutant de x tirée d'une gaussienne centrée sur x et d'écart-type σ
5. un mutant de x centré sur x avec un bruit uniforme dans $[-1, +1]$

init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

for i in range(evaluations):

$x' = x + \sigma N(0, I)$

if $\text{fitness}(x') \geq \text{fitness}(x)$:

$x = x'$

$\sigma = 2 * \sigma$

else:

$\sigma = 2^{-1/4} * \sigma$

} the “one fifth” rule

init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

for i in range(evaluations):

$x' = x + \sigma N(0, I)$

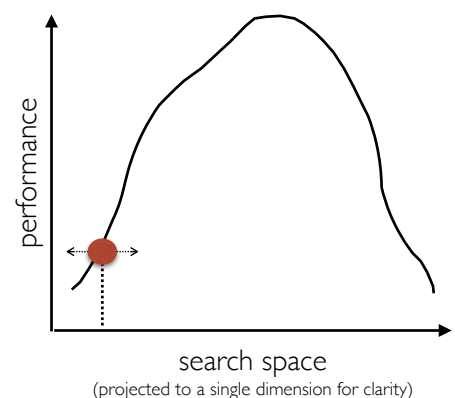
if $\text{fitness}(x') \geq \text{fitness}(x)$:

$x = x'$

$\sigma = 2 * \sigma$

else:

$\sigma = 2^{-1/4} * \sigma$



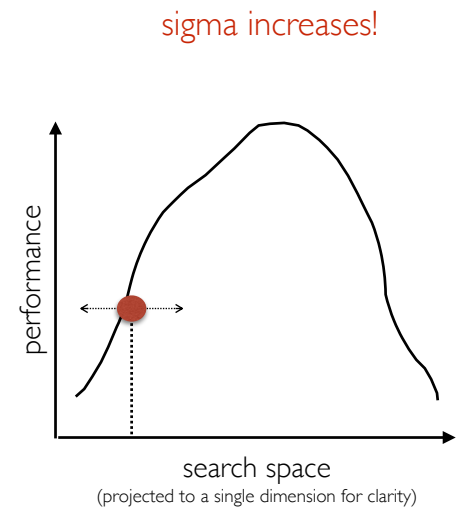
init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

```

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 
         $\sigma = 2 * \sigma$ 
    else:
         $\sigma = 2^{-1/4} * \sigma$ 

```



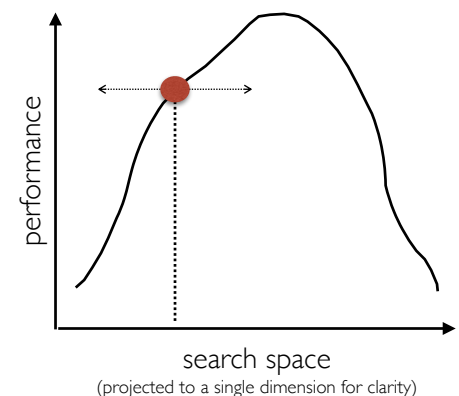
init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

```

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 
         $\sigma = 2 * \sigma$ 
    else:
         $\sigma = 2^{-1/4} * \sigma$ 

```

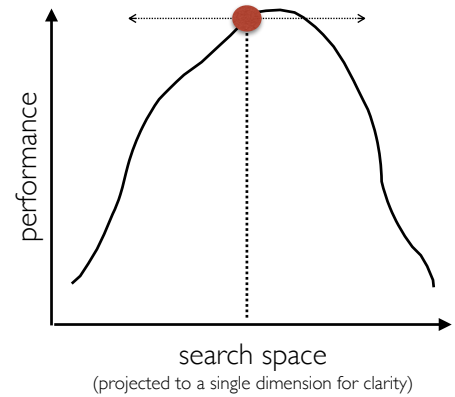


init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

```

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 
         $\sigma = 2 * \sigma$ 
    else:
         $\sigma = 2^{-1/4} * \sigma$ 
  
```

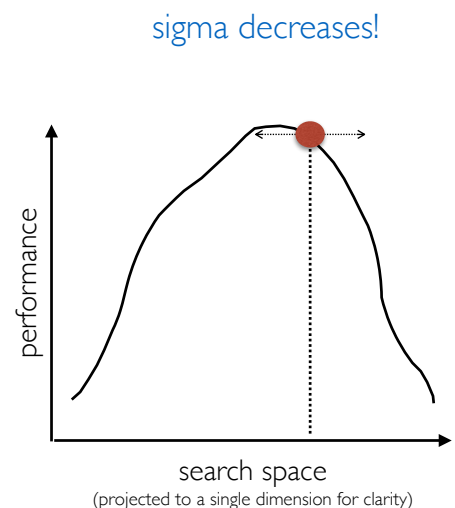


init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

```

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 
         $\sigma = 2 * \sigma$ 
    else:
         $\sigma = 2^{-1/4} * \sigma$ 
  
```



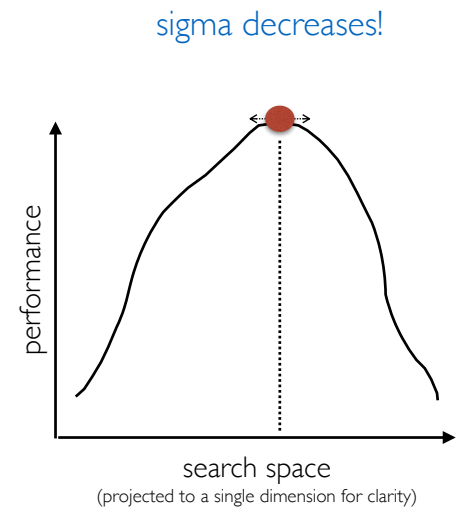
init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

```

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 
         $\sigma = 2 * \sigma$ 
    else:
         $\sigma = 2^{-1/4} * \sigma$ 

```



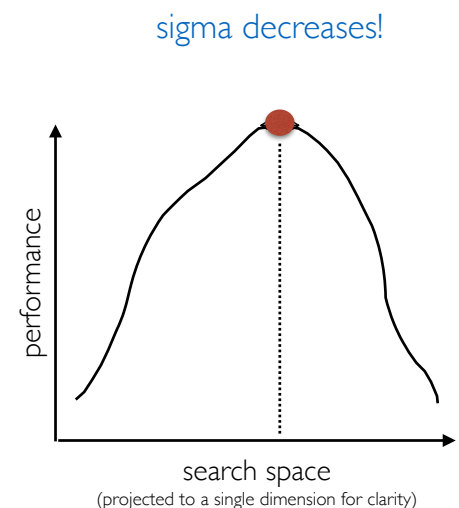
init: $x \in \mathbb{R}^n$

init: $\sigma > 0$

```

for i in range(evaluations):
     $x' = x + \sigma N(0, I)$ 
    if fitness( $x'$ )  $\geq$  fitness( $x$ ):
         $x = x'$ 
         $\sigma = 2 * \sigma$ 
    else:
         $\sigma = 2^{-1/4} * \sigma$ 

```



adaptative convergence

- (1+1)-ES algorithm with 1/5th rule
 - adaptive convergence
 - limited to convex functions
 - sensitive to noise —> hint: perform more evaluations!

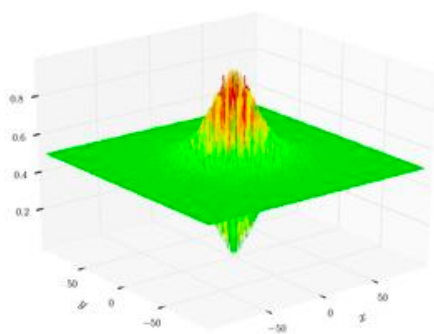
Evolution Strategies

$(\mu=1, \lambda=n)$ -ES algorithm

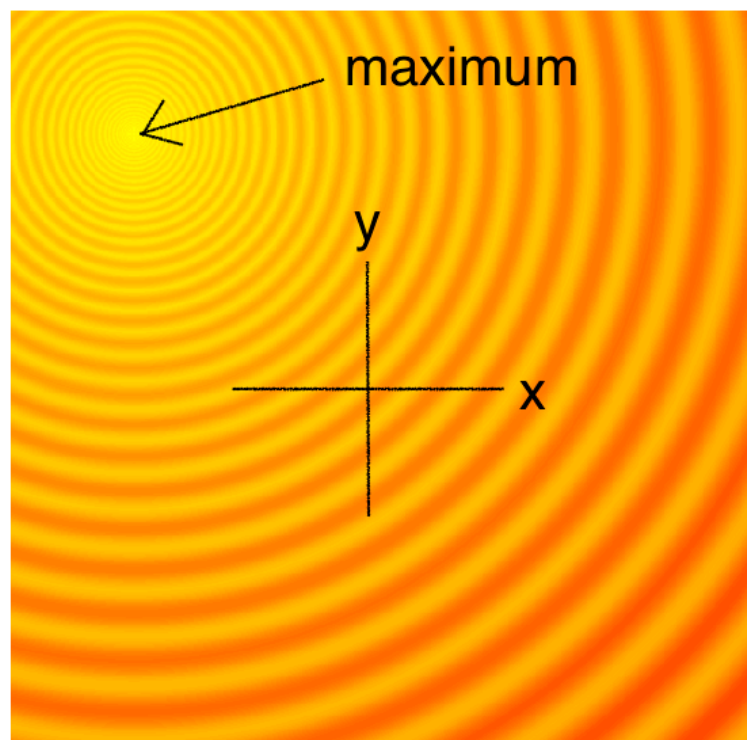
- (1,n)-ES algorithm
 - create n individuals using gaussian noise
 - select the *best* individual
 - start again

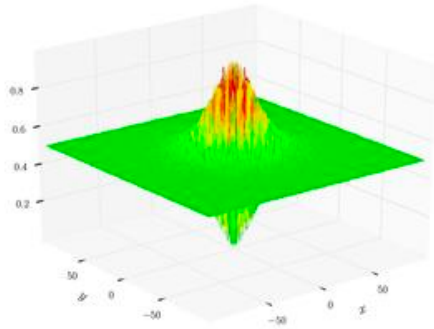
Schaffer-2D function

40

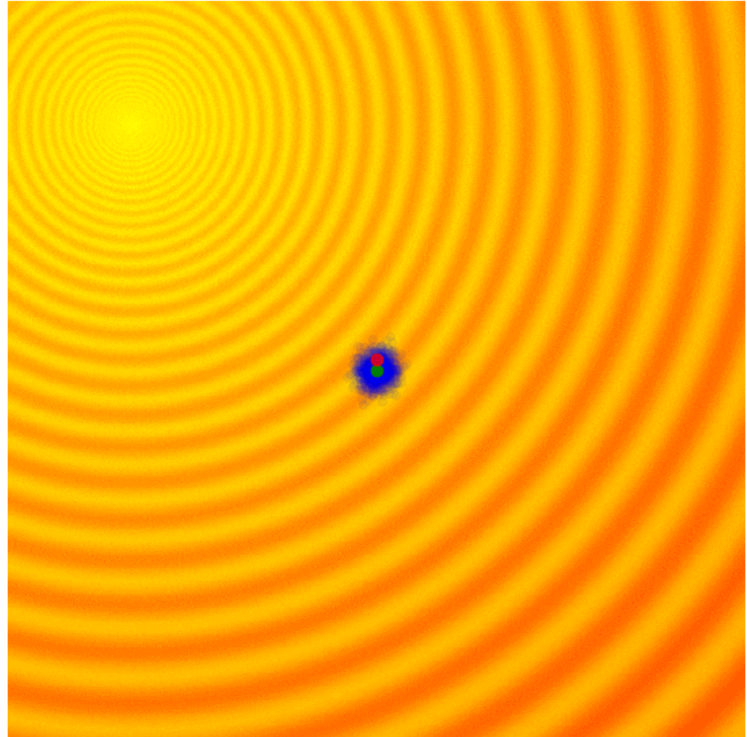


$$f(x, y) = 0.5 + \frac{\sin^2(x^2 - y^2) - 0.5}{[1 + 0.001(x^2 + y^2)]^2}$$





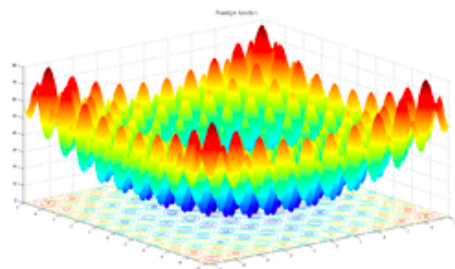
$$f(x, y) = 0.5 + \frac{\sin^2(x^2 - y^2) - 0.5}{[1 + 0.001(x^2 + y^2)]^2}$$



wikipedia

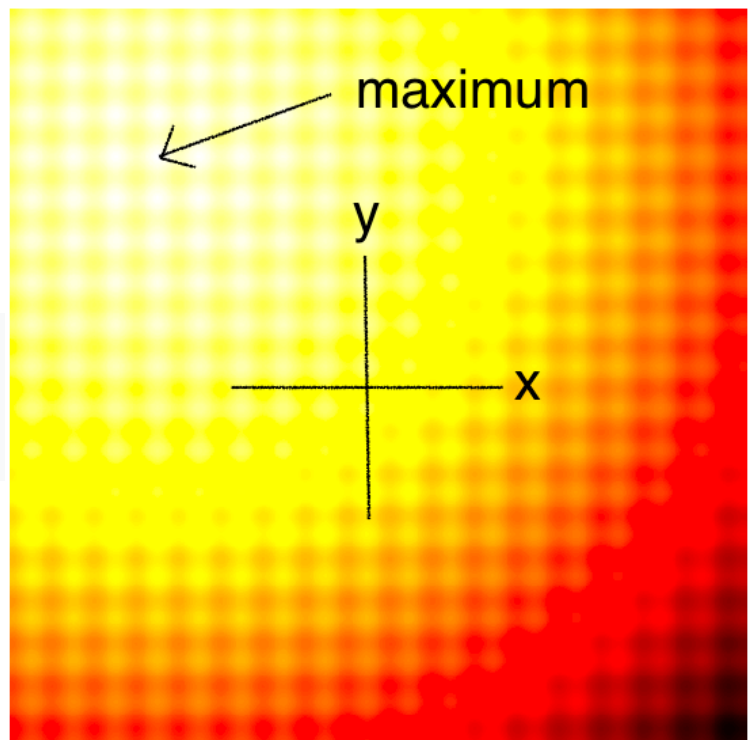
Ha, D. (2017) A Visual Guide to Evolution Strategies, blog.otoro.net, 2017

Rastrigin-2D function



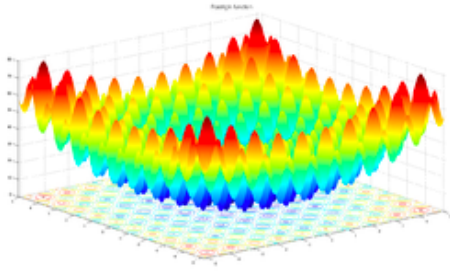
$$f(\mathbf{x}) = An + \sum_{i=1}^n [x_i^2 - A \cos(2\pi x_i)]$$

where: $A = 10$



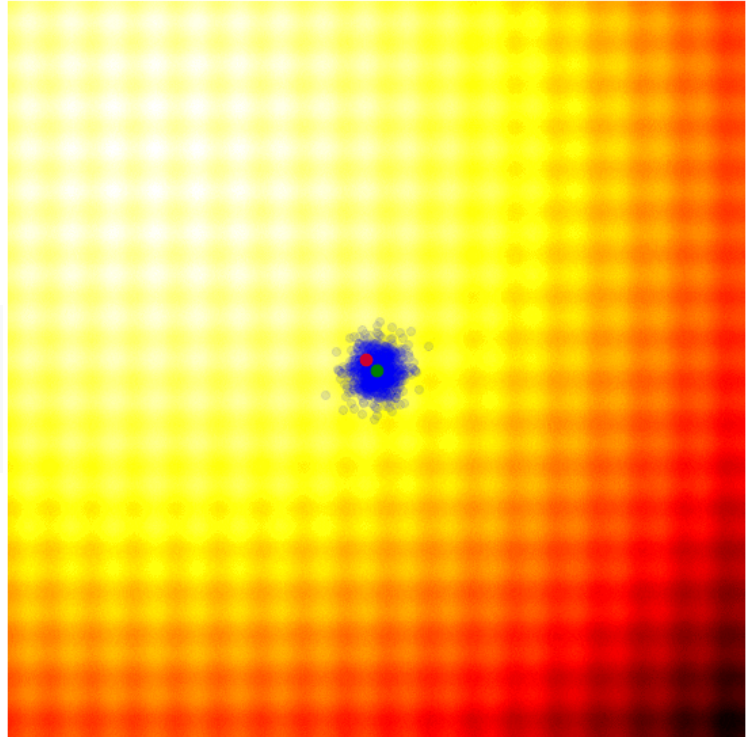
wikipedia

Ha, D. (2017) A Visual Guide to Evolution Strategies, blog.otoro.net, 2017



$$f(\mathbf{x}) = An + \sum_{i=1}^n [x_i^2 - A \cos(2\pi x_i)]$$

where: $A = 10$



wikipedia

Ha, D. (2017) A Visual Guide to Evolution Strategies, blog.otoro.net, 2017

- (1,n)-ES algorithm
 - avoid local optima ...to some extent
 - slow convergence, fixed step-size
 - exploit only the single best individual... natural extension $\mu > 1$

Evolution Strategies

(μ, λ) -ES and $(\mu + \lambda)$ -ES algorithms

(μ, λ) -ES and $(\mu + \lambda)$ -ES algorithms

46

- Algorithm
 - select the best μ individuals (parents)
 - create λ individuals (offspring)
 - ▶ to build a new individual out of λ :
 - take a random individual among the μ parents
 - apply a mutation to get the new child (e.g. gaussian mutation)
 - build the new population
 - ▶ (μ, λ) -ES method: the new population contains only λ individuals
 - ▶ $(\mu + \lambda)$ -ES method: the new population contains both $\mu + \lambda$ individuals
 - continue

With $\mu < \lambda$

Q: μ désigne...

1. le nombre de parents
2. le nombre d'enfants
3. la taille de la population

Q: λ désigne...

1. le nombre de parents
2. le nombre d'enfants

Q: si la fonction objectif est convexe, il vaut mieux utiliser...

1. $(1+1)$ -ES
2. $(1, \lambda=10)$ -ES

Q: l'algorithme $(\mu=4, \lambda=1)$ -ES permet...

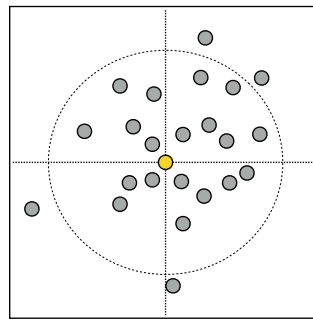
1. d'optimiser une fonction uni-modale
2. d'optimiser une fonction multimodale
3. de faire n'importe quoi

Q: une matrice de covariance...

1. je sais ce que c'est
2. je ne sais pas ce que c'est

Evolution Strategies

Probability-weighted averaging method



$$(1+1)\text{-ES} \begin{cases} x' = m + \sigma \mathcal{N}(0, I) \\ \text{if } f(x) > f(m) : m \leftarrow x \end{cases}$$

$$(\mu/\mu, \lambda)\text{-ES} \begin{cases} x' = m + \sigma \mathcal{N}_i(0, I) \\ m \leftarrow \sum_{i=1}^{\mu} w_i x_{i:\lambda} \end{cases}$$

μ/μ : recombination of all parents

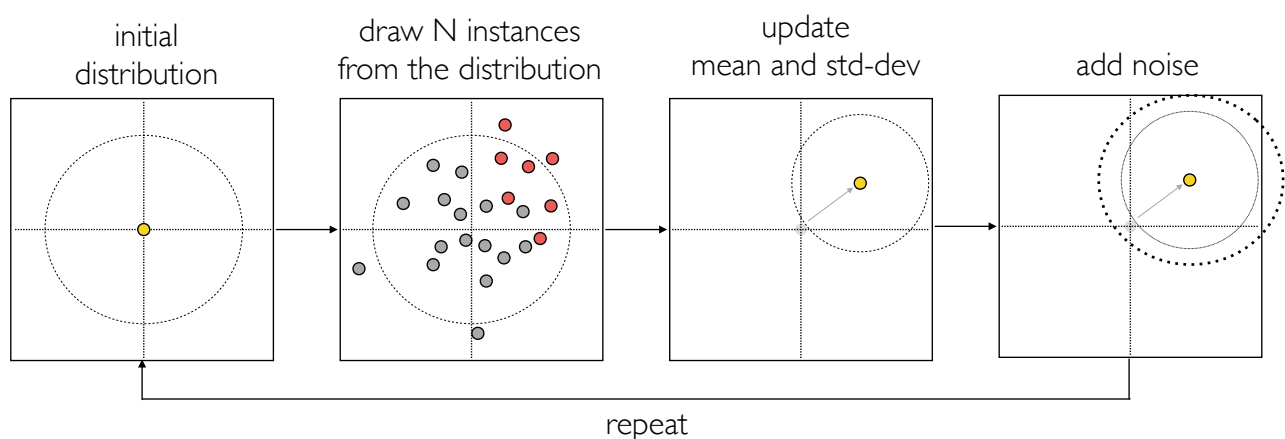
→ provides a **single parental centroid**

identity matrix: dimension-specific mutation step-size

remark: $W_i = 1.0$ (default) or < 1.0 to normalize wrt fitness values

Cross-Entropy Method for ES (CEM-ES) — simplified

54



● Probability-weighted averaging method

► Elements

- m : mean of distribution
- σ^2 : standard deviation

► Principles

- update m and σ^2 by sampling λ , and with selection pressure μ

- Cross-Entropy Method with $N(0, I)$
 - the whole population is used (not just the best)
 - better at avoiding local optima
 - adaptive convergence
 - not great with non-separable problems (identity matrix, again)

Evolution Strategies

Multivariate normal distribution

Isotropic Gaussian Distribution

$$\sigma\mathcal{N}(O, I) = \begin{bmatrix} \sigma^2 & 0 & \cdots & 0 \\ 0 & \sigma^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma^2 \end{bmatrix}$$

...as used
until now

Diagonal Gaussian Distribution

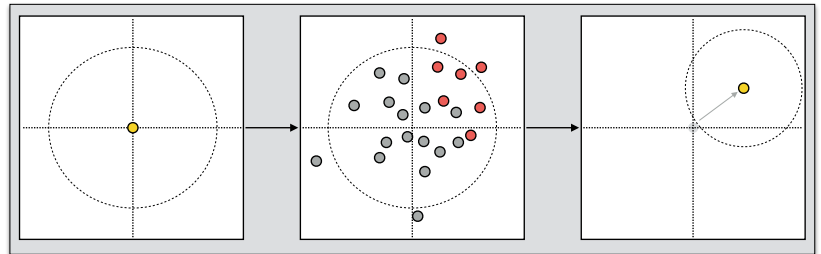
$$\mathcal{N}(O, D^2) = \begin{bmatrix} \sigma_1^2 & 0 & \cdots & 0 \\ 0 & \sigma_2^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_n^2 \end{bmatrix}$$

General Multivariate Gaussian Distribution

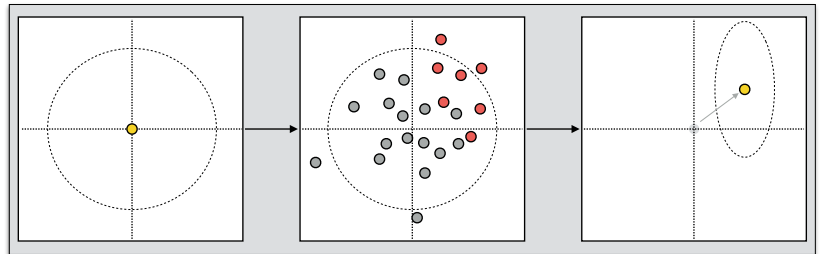
$$\mathcal{N}(O, C) = \begin{bmatrix} \sigma_1^2 & \sigma_1\sigma_2\rho_{12} & \cdots & \sigma_1\sigma_n\rho_{1n} \\ \sigma_2\sigma_1\rho_{21} & \sigma_2^2 & \cdots & \sigma_2\sigma_n\rho_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_n\sigma_1\rho_{n1} & \sigma_n\sigma_2\rho_{n2} & \cdots & \sigma_n^2 \end{bmatrix}$$

$\mathcal{N}(O, D^2)$ — D^2 instead of D to emphasise variance (both notations are correct, but D^2 emphasises variance over standard deviation)

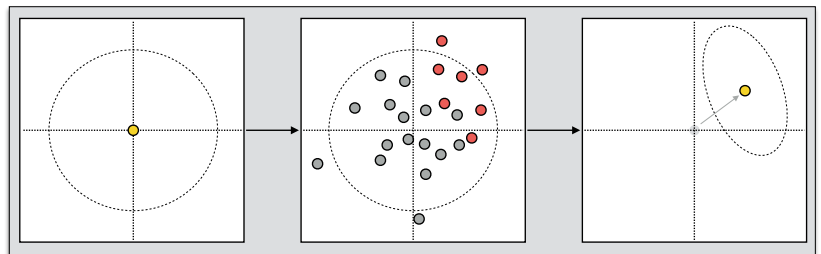
$$(\mu/\mu, \lambda)\text{-ES} \begin{cases} x' = m + \sigma\mathcal{N}_i(0, I) \\ m \leftarrow \sum_{i=1}^{\mu} w_i x_{i:\lambda} \end{cases}$$



$$(\mu/\mu, \lambda)\text{-ES} \begin{cases} x' = m + \sigma\mathcal{N}_i(0, D^2) \\ m \leftarrow \sum_{i=1}^{\mu} w_i x_{i:\lambda} \end{cases}$$

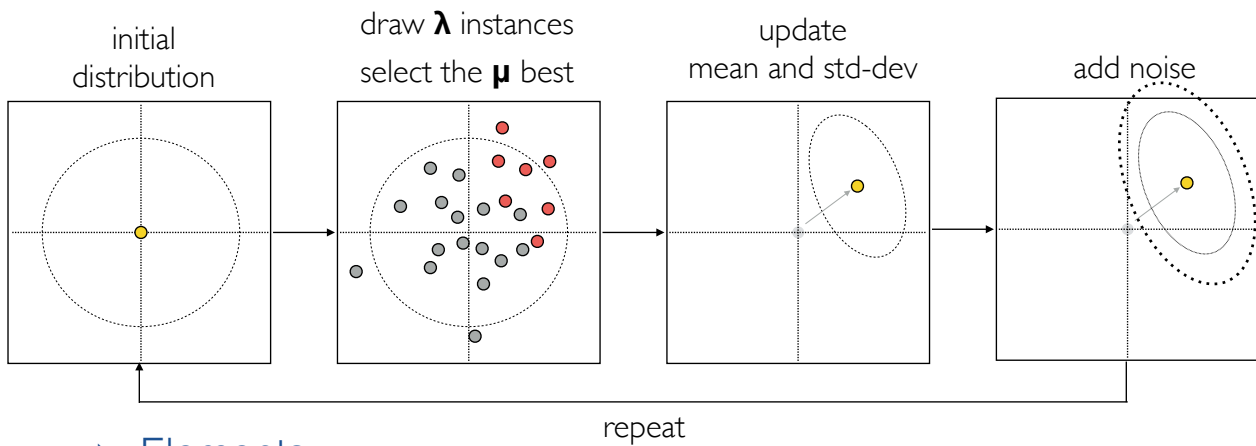


$$(\mu/\mu, \lambda)\text{-ES} \begin{cases} x' = m + \sigma\mathcal{N}_i(0, C) \\ m \leftarrow \sum_{i=1}^{\mu} w_i x_{i:\lambda} \end{cases}$$



σ as a prefix means global control parameter — could be included in D and C , but handy to have it as a separate parameter.

Full algorithm with **Covariance matrix**



► Elements

- m : mean of distribution
- σ^2 : variance
- ϵ : noise

► Principles

- update m and C by sampling λ , and with selection pressure μ
- noise (ϵ) decreases over time ("noise damping")

Elitism: add the best-ever to the μ current best solutions

QUIZZ

Q: CEM utilise...

1. toutes les solutions candidates
2. les N meilleures solutions candidates
3. les N meilleures solutions candidates pondérées par leur performance

Q: Après avoir mis à jour la distribution, l'algorithme doit...

1. mettre à jour la matrice identité
2. mettre à jour la matrice de covariance
3. appliquer un bruit à la matrice
4. tirer au hasard N points de la distribution

Q: Par rapport à $\sigma^2 N(0, I)$, une matrice de covariance permet...

1. d'être robuste aux rotations
2. de prendre en compte l'éparpillement des solutions candidates
3. d'exploiter l'indépendance entre dimensions

Evolution Strategies

CMA-ES algorithm

64

$$(\mu/\mu, \lambda)\text{-ES} \begin{cases} x' = m + \sigma \mathcal{N}_i(0, \mathbf{C}) \\ m \leftarrow \sum_{i=1}^{\mu} w_i x_{i:\lambda} \end{cases}$$

similar to CEM

C: covariance matrix, $\mathbf{C} \in \mathbb{R}^{n \times n}$

● Co-variance Matrix Adaptation Evolution Strategy

- ▶ self-adaptive search, (nearly) parameter-free
- ▶ exploit temporal correlations (i.e. may move faster in one direction)
- ▶ invariance to
 - order-preserving transformations (only depends on rank)
 - scale (if initial step-size σ and initial search point chosen accordingly)
 - scaling of variable (diagonal invariance)
 - angle preserving transformations (rotation, reflection, translation)
 - invertible linear transformation
- ▶ works in large dimension (100+)

Hansen, Ostermeier (2001)

Covariance matrix

66

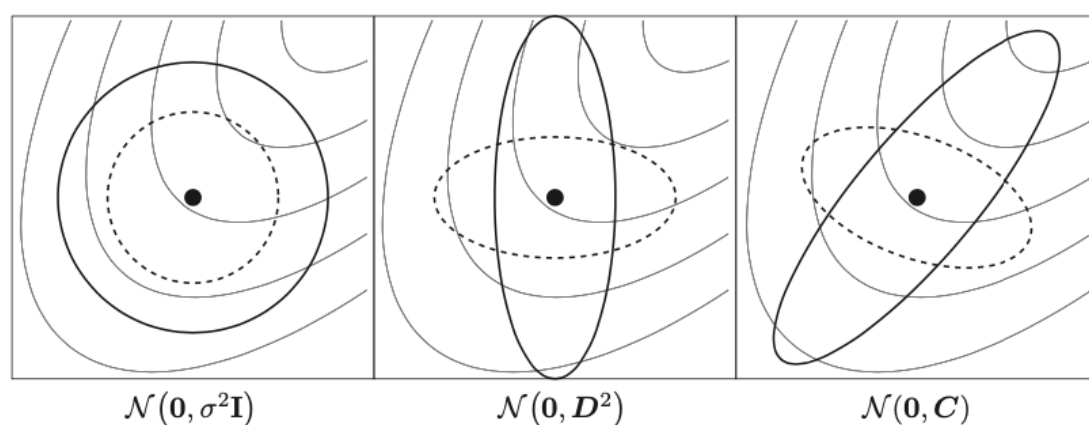
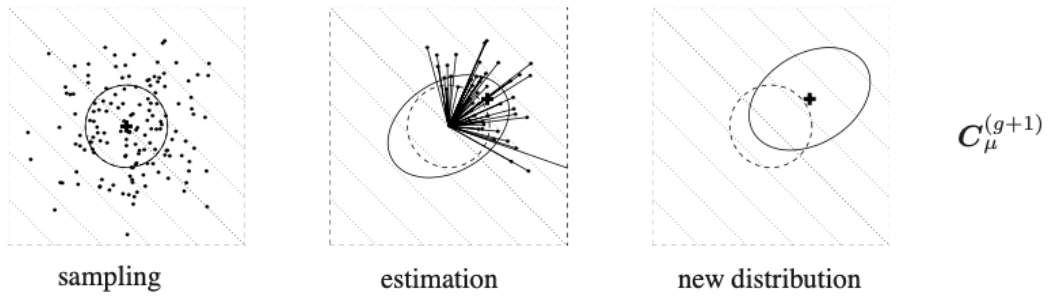


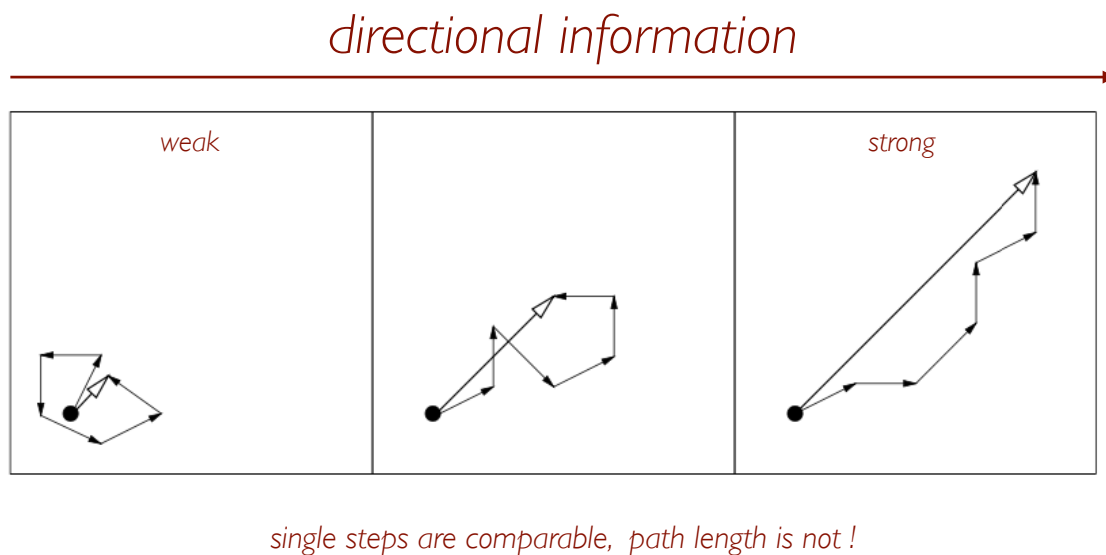
Figure 1: Ellipsoids depicting one- σ lines of equal density of six different normal distributions, where $\sigma \in \mathbb{R}_{>0}$, $\mathbf{D}$ is a diagonal matrix, and $\mathbf{C}$ is a positive definite full covariance matrix. Thin lines depict possible objective function contour lines



Hansen (2016) The CMA Evolution Strategy: a Tutorial. Arxiv.

Correlation over generations

68



→ CMAES exploits directional information over generations

Hansen (2016) The CMA Evolution Strategy: a Tutorial. Arxiv.

$$\mathbf{C}^{(g+1)} = \underbrace{(1 - c_1 - c_\mu \sum w_j) \mathbf{C}^{(g)}}_{\text{Covariance decay}} + \underbrace{c_1 \mathbf{p}_c^{(g+1)} (\mathbf{p}_c^{(g+1)})^T}_{\text{Rank-one update}} + \underbrace{c_\mu \sum_{i=1}^{\lambda} w_i \mathbf{y}_{i:\lambda}^{(g+1)} (\mathbf{y}_{i:\lambda}^{(g+1)})^T}_{\text{Rank-}\mu \text{ update}}$$

Covariance decay

How much the previous $\mathbf{C}$ is scaled down
decay = 0 means previous $\mathbf{C}$ is forgotten

Rank-one update

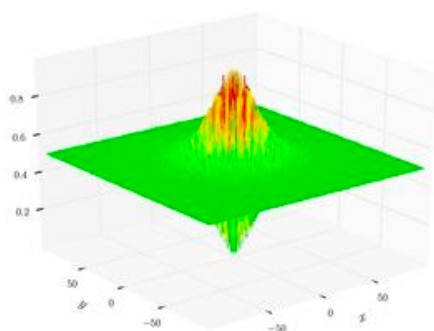
Uses directional movement over generations
→ step-size based

Rank- μ update

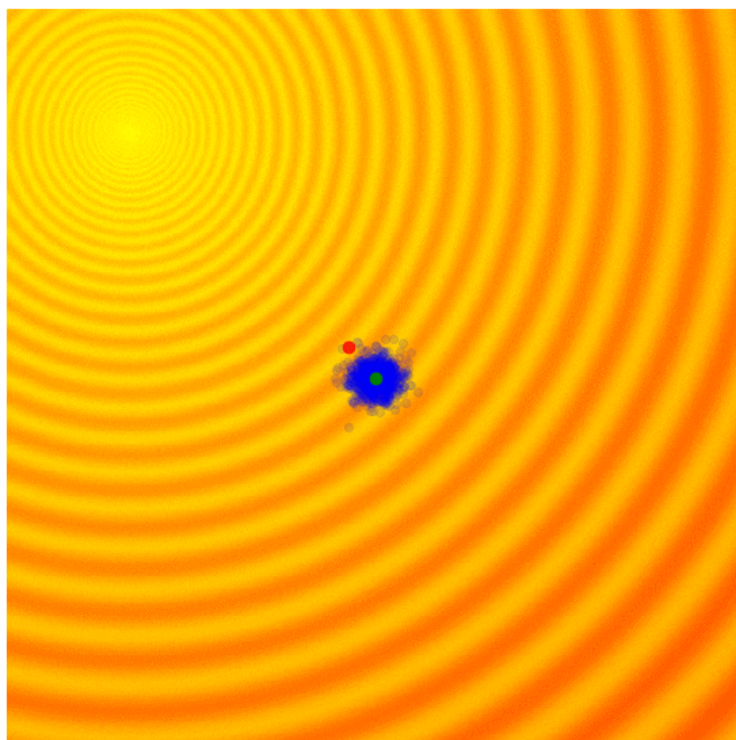
Uses current selected individual
→ sampling based
(similar to earlier ES)

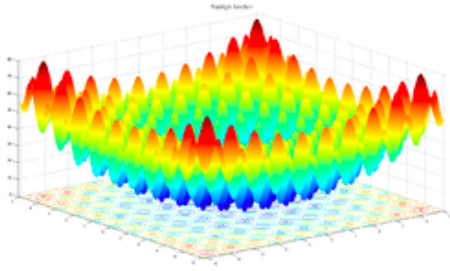
with c_1 and c_μ used to balance between terms

Schaffer-2D function



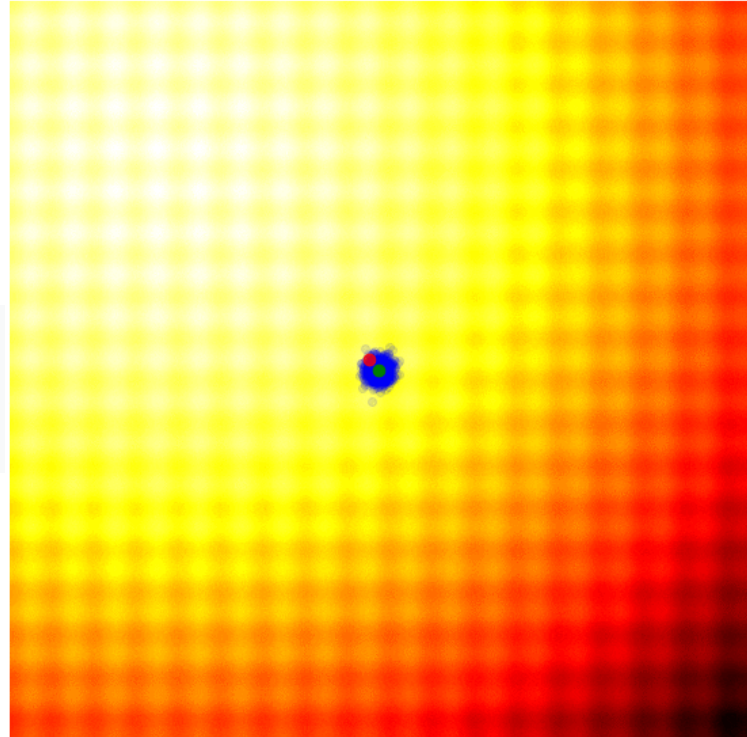
$$f(x, y) = 0.5 + \frac{\sin^2(x^2 - y^2) - 0.5}{[1 + 0.001(x^2 + y^2)]^2}$$





$$f(\mathbf{x}) = An + \sum_{i=1}^n [x_i^2 - A \cos(2\pi x_i)]$$

where: $A = 10$



wikipedia

Ha, D. (2017) A Visual Guide to Evolution Strategies, blog.otoro.net, 2017

CMA-ES : summary

72

Input: $\mathbf{m} \in \mathbb{R}^n$, $\sigma \in \mathbb{R}_+$, λ

Initialize: $\mathbf{C} = \mathbf{I}$, and $\mathbf{p}_c = \mathbf{0}$, $\mathbf{p}_\sigma = \mathbf{0}$,

Set: $c_c \approx 4/n$, $c_\sigma \approx 4/n$, $c_1 \approx 2/n^2$, $c_\mu \approx \mu_w/n^2$, $c_1 + c_\mu \leq 1$, $d_\sigma \approx 1 + \sqrt{\frac{\mu_w}{n}}$,
and $w_{i=1 \dots \lambda}$ such that $\mu_w = \frac{1}{\sum_{i=1}^{\mu} w_i^2} \approx 0.3 \lambda$

While not terminate

$\mathbf{x}_i = \mathbf{m} + \sigma \mathbf{y}_i$, $\mathbf{y}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{C})$, for $i = 1, \dots, \lambda$

$\mathbf{m} \leftarrow \sum_{i=1}^{\mu} w_i \mathbf{x}_{i:\lambda} = \mathbf{m} + \sigma \mathbf{y}_w$ where $\mathbf{y}_w = \sum_{i=1}^{\mu} w_i \mathbf{y}_{i:\lambda}$

$\mathbf{p}_c \leftarrow (1 - c_c) \mathbf{p}_c + \mathbb{1}_{\{\|\mathbf{p}_\sigma\| < 1.5\sqrt{n}\}} \sqrt{1 - (1 - c_c)^2} \sqrt{\mu_w} \mathbf{y}_w$

$\mathbf{p}_\sigma \leftarrow (1 - c_\sigma) \mathbf{p}_\sigma + \sqrt{1 - (1 - c_\sigma)^2} \sqrt{\mu_w} \mathbf{C}^{-\frac{1}{2}} \mathbf{y}_w$

$\mathbf{C} \leftarrow (1 - c_1 - c_\mu) \mathbf{C} + c_1 \mathbf{p}_c \mathbf{p}_c^T + c_\mu \sum_{i=1}^{\mu} w_i \mathbf{y}_{i:\lambda} \mathbf{y}_{i:\lambda}^T$

$\sigma \leftarrow \sigma \times \exp\left(\frac{c_\sigma}{d_\sigma} \left(\frac{\|\mathbf{p}_\sigma\|}{\mathbb{E}\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|} - 1\right)\right)$

sampling

update mean

cumulation for $\mathbf{C}$

cumulation for σ

update $\mathbf{C}$

update of σ

→ Sampling and
mean adaptation

→ Directional mouvement
adaptation

→ Covariance & step size
adaptation

- CMA-ES
 - the whole population is used
 - adaptive convergence
 - avoid local optima
 - **adaptive exploration mechanisms**
 - ...no true global search. Using a restart mechanism can do the trick.
 - ...computing covariance matrix does not scale well (easy fix: switch back to $N(0, D^2)$)

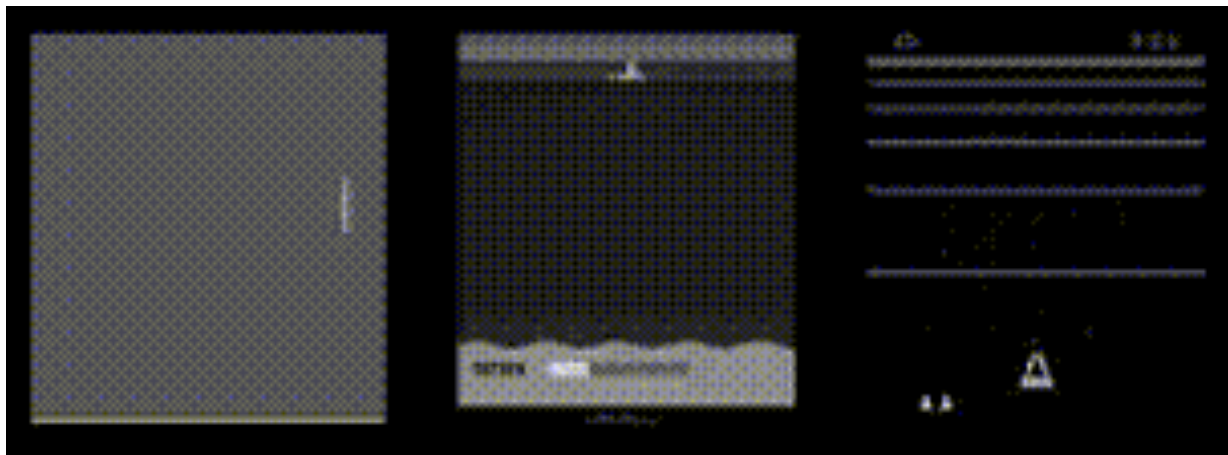
Many implementations, for example <https://deap.readthedocs.io/en/master/examples/cmaes.html>

General resources: https://cma-es.github.io/cmaes_sourcecode_page.html



Evolutionary Strategies

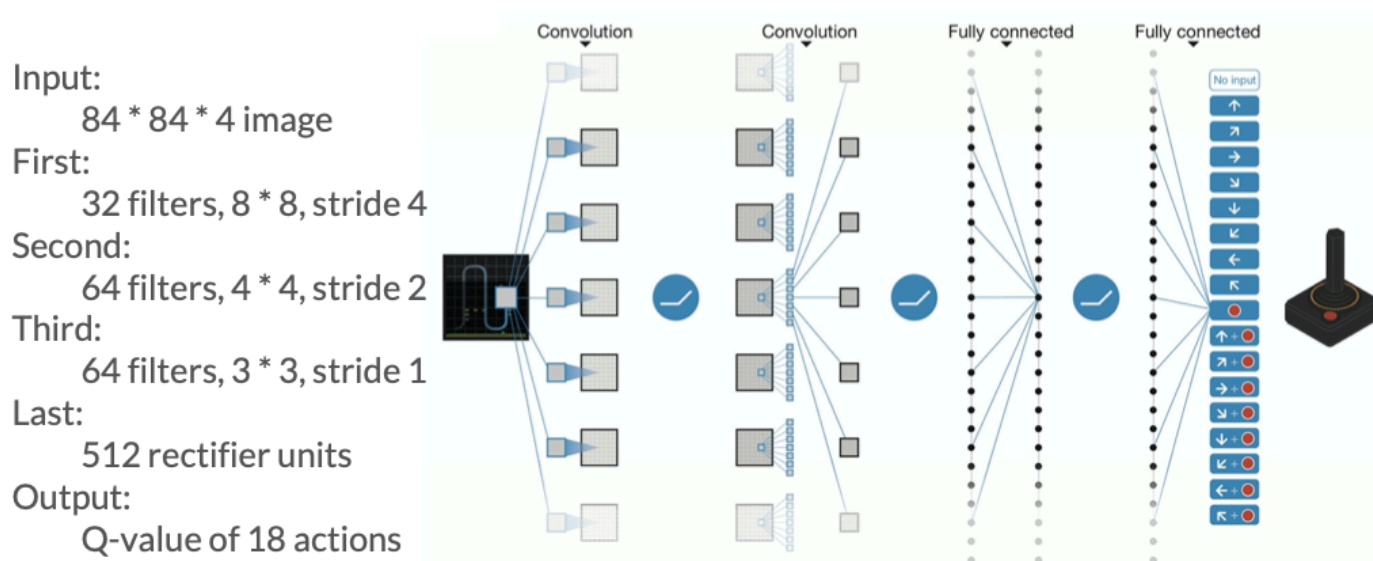
Case study



51 Atari 2600 games

A very popular benchmark since 2016

Mnih et al. (2016) — Google Deepmind ; Salimans et al. (2017) — OpenAI



~50 millions weights, 2 millions are learned

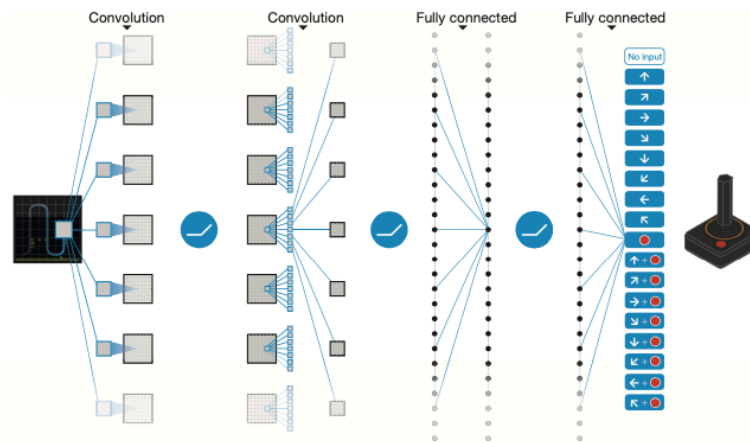


Figure 1 | Schematic illustration of the convolutional neural network. The details of the architecture are explained in the Methods. The input to the neural network consists of an $84 \times 84 \times 4$ image produced by the preprocessing map ϕ , followed by three convolutional layers (note: snaking blue line

symbolizes sliding of each filter across input image) and two fully connected layers with a single output for each valid action. Each hidden layer is followed by a rectifier nonlinearity (that is, $\max(0, x)$).

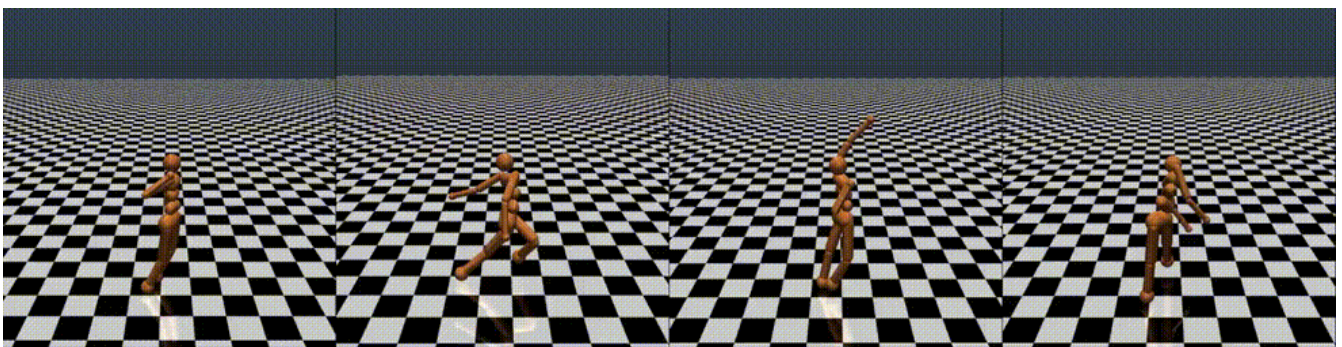
→ ~50 millions NN weights, 2 millions are learned

- Control: A3C, 320 million frames, ~1 day [Mnih et al., 2016]
- Result: ES better on 23 games, worse on 28 games
- Budget per game for ES: 1 billion frames
 - ▶ ES does not compute backprop nor value function => same time
 - ▶ ES easier to parallelise: swap fitness, not gradient => ~1 hour

Mnih et al. (2016) — Google Deepmind

MuJoCo tasks — Walker2D

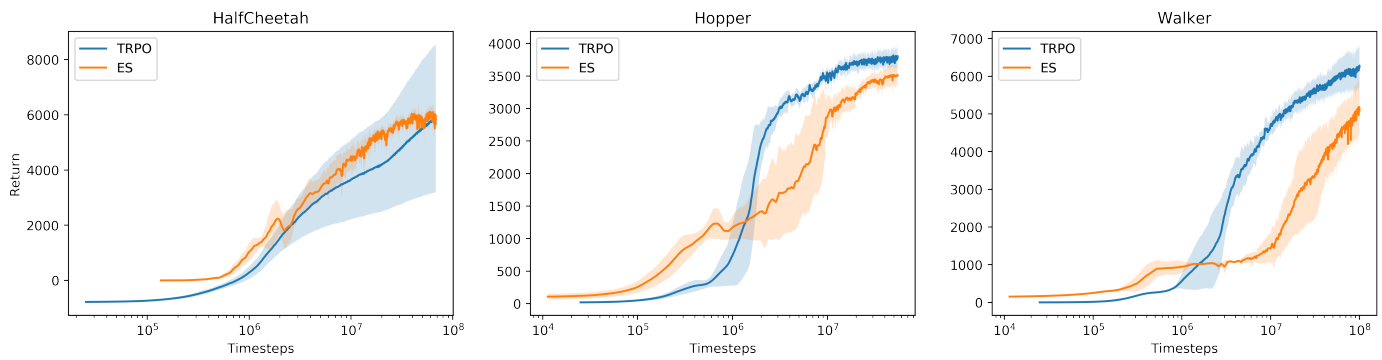
78



- multilayer perceptrons
 - ▶ 2x 64-unit hidden layers
 - ▶ Non-linear activation function (hyperbolic tangent)

Also tested with *HalfCheetah*, *Hopper*, *InvertedDoublePendulum*, *InvertedPendulum* and *Swimmer*

Salimans et al. (2017) — OpenAI



• Results

- ▶ Similar performance after 5 million time steps
- ▶ ES requires more samples (3x to 10x)*
- ▶ ES requires less computational effort (~3x, do not use backprop)
- ▶ ES finds more diverse solutions (original gaits)

*

10x more samples in "hard" environments (Walker3D, Hopper)

3x more samples in "simple" environments (HalfCheetah, InvertedDoublePendulum, InvertedPendulum and Swimmer)

Time steps: ES benefits from parallelisation, so 1 time step = N interactions Salimans et al. (2017) — OpenAI

80

- MuJoCo robotics tasks
- Use neural networks with <300 parameters
 - ▶ other method use 13x more parameters

- Comparing ES and state-of-the-art RL methods
 - ▶ ES is competitive with state-of-the-art RL methods
 - ▶ ES explores more diverse solutions
 - ▶ ES explores more robust solutions
 - ▶ ES requires much more samples (3x, 10x) than RL methods
 - ▶ But...
 - ES is faster (3x time net gain, incl. benefits (no backprop, parallel.) and costs (lower sample efficiency))
 - ES is highly scalable (only fitness values exchanged, linear cost for parallelisation)
 - ES needs no hyper-parameter tuning (hidden cost for gradient-based method)
 - ▶ And also...
 - ES is ok w/ sparse or delayed rewards, and long-time horizons
 - ES may not require large ANNs (gradient policy search needs gradients, not ES)

Further reading: <https://eng.uber.com/deep-neuroevolution/>

Conclusions

- Evolutionary Strategies...
 - as a blackbox optimisation method
 - ▶ performs derivative-free optimisation
 - ▶ addresses non-convex problems
 - ▶ ...and can be implemented in a highly parallel fashion
 - relevant for robotics
 - ▶ ...but also for many other problems